

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I K DHANUSH SRIVATSAN / 192524503(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



—

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock.

Suggest an optimisation to eliminate the deadlock without changing process logic.

(10 Marks)

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready.

You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

(10 Marks)

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory map shows:

- Code: 5 MB

- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations.

(10 Marks)

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 =

FCFS P5 is in Q3 but keeps getting postponed.

Debug the root cause of starvation and suggest an optimisation to ensure fairness.

(10 Marks)

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead.

Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead.

(10 Marks)

Q6. Semaphore Synchronization Debugging

A producer-consumer application occasionally hangs during execution. The semaphore values observed are:

mutex = 1

empty = 0

full = 10

Buffer size = 10

Producer Code:

wait(mutex)

wait(empty)

produce item

signal(full)

signal(mutex)

Consumer Code:

wait(mutex)

wait(full)

consume item

signal(empty)

signal(mutex)

Identify the synchronization bug causing the system to hang.

Explain which deadlock condition is being violated.

Suggest an optimization to ensure correct synchronization without changing the producer-consumer functionality. (10 Marks)

Q7. Thrashing Detection and Optimization

A virtual memory system reports the following statistics:

Number of processes: 50

Average page fault rate: 250 faults/sec

CPU Utilization: 18%

Disk Utilization: 95%

Free Frames Available: 2%

The operating system suspects thrashing.

Debug the root cause of the performance degradation.

Explain the relationship between page faults and CPU utilization.

Propose two operating system optimizations to reduce thrashing. (10 Marks)

Q8. File System Performance Debugging

A database server experiences slow file access times. System logs indicate:

Average file size: 500 MB

Disk seeks per request: 120

Disk utilization: 92%

Cache hit ratio: 15%

Identify the file system bottleneck affecting performance.

Explain why disk access latency is increasing.

Suggest two optimizations to improve file access efficiency. (10 Marks)

Q9. Thread Scheduling Debugging

A web server creates one thread per client request. During peak hours:

Active Threads: 5,000

CPU Utilization: 60%

Memory Usage: 92%

Average Response Time: 12 seconds

Scheduler logs show frequent thread creation and termination.

Identify the primary issue affecting server performance.

Explain how thread management contributes to the delay.

Recommend two optimizations to improve scalability and response time. (10 Marks)

Q10. Disk Scheduling Optimization Problem

A disk scheduling log shows the following request queue:

98, 183, 37, 122, 14, 124, 65, 67

Current Head Position: 53

The system uses FCFS scheduling and reports:

Average seek time: Very High

Throughput: Low

User complaints regarding slow responseDebug why FCFS is causing poor performance in this scenario.

Compare FCFS with one suitable disk scheduling algorithm for this workload.

Propose an optimization to improve seek time and overall throughput. (10 Marks)

OS Debugging Answers

Q1. Deadlock Debugging

Step-by-step trace of what happens:

Time	P1	P2
t=0	lock(R1) → acquires R1	lock(R2) → acquires R2
t=0 to 100ms	waiting	waiting
t=100ms	lock(R2) → blocks (R2 held by P2)	lock(R1) → blocks (R1 held by P1)
forever	blocked	blocked

Both processes deliberately introduce a 100ms delay between acquiring their first lock and requesting the second. This window guarantees the *other* process has time to grab its own first lock — making the deadlock almost certain to occur on every run, not just a rare race condition.

All four Coffman conditions, explained in context:

1. **Mutual exclusion** — R1 and R2 are lockable resources, so only one process can hold each at a time (this is inherent to using locks, not a bug by itself).
2. **Hold and wait** — P1 holds R1 while waiting for R2; P2 holds R2 while waiting for R1. Neither releases what it already has while waiting for more.
3. **No preemption** — the OS/runtime cannot forcibly strip R1 from P1 to give it to P2; locks are only released voluntarily via unlock().
4. **Circular wait** — P1 → waits on R2 → held by P2 → waits on R1 → held by P1. This closes the cycle.

Deadlock requires **all four** simultaneously — removing any one breaks it. Since mutual exclusion and no-preemption are usually intrinsic to how locks work, the practical fixes almost always target **hold-and-wait** or **circular wait**.

Optimisation in depth — Lock Ordering (breaks circular wait):

Assign a total order to resources, e.g., $R1 < R2$. Require every process to acquire locks in ascending order regardless of what it needs to do with them:

P1: lock(R1) → wait(100ms) → lock(R2) → critical section → unlock(R2) → unlock(R1)

P2: lock(R1) → wait(100ms) → lock(R2) → critical section → unlock(R2) → unlock(R1)

Now P2 also requests R1 first. Whichever process gets R1 first will proceed to also get R2 (since the other process is stuck waiting on R1, it never touches R2), so no cycle can form. This is called the **resource-ordering / hierarchy protocol**, and it's the standard textbook (Coffman) solution — it doesn't change what work each process does inside its critical section, only the *order* in which locks are requested, which is what "without changing process logic" is asking for.

Alternative approaches worth mentioning:

- **Try-lock with timeout + backoff:** try_lock(R2, timeout=50ms); if it fails, release R1 and retry after a random delay. This avoids deadlock at the cost of occasional wasted work (livelock risk if not randomized).
 - **Wait-die / Wound-wait schemes** (used in databases): assign timestamps to processes; older processes either force younger ones to release resources or wait, ensuring no cycle can form.
-

Q2. CPU Utilisation Problem

Why the ready↔waiting cycling matters:

CPU utilisation is fundamentally: $\text{Utilisation} = 1 - P(\text{all } n \text{ processes are waiting on I/O simultaneously})$. If each process spends most of its time waiting (I/O-bound behavior), and the degree of multiprogramming (n) is too low, the probability that *all* processes are blocked at once rises sharply — leaving the CPU idle even though, on average, processes do become "ready" periodically.

Concretely: if a process spends 80% of its time waiting on I/O and only 20% computing, and you only have 2–3 such processes loaded, there's a high chance both are blocked at the same instant. With $\text{utilisation} = 1 - (0.8)^n$, at $n=2$ that's only ~36% utilisation — matching the ~30% observed.

Deeper diagnosis path:

- Check **degree of multiprogramming** — how many processes are resident in memory ready to run.
- Check **burst-time distribution** — are these predominantly short CPU bursts followed by I/O (I/O-bound), or long CPU bursts (CPU-bound)?

- Check whether the scheduler is blocking on **synchronous I/O calls** that could instead be asynchronous.

Optimisations, elaborated:

1. **Raise the multiprogramming degree** — load more processes into memory (subject to available RAM) so when one blocks, others are ready to run. This directly reduces P(all blocked).
 2. **Multilevel Feedback Queue (MLFQ)** — dynamically classify processes: those that block quickly (I/O-bound) get high priority for quick re-dispatch (keeping I/O devices busy), while CPU-bound processes fill in the gaps. This maximizes overlap between CPU computation and I/O operations for different processes.
 3. **Asynchronous I/O / non-blocking syscalls** — instead of a process blocking the whole time waiting for I/O completion, it issues the I/O request and continues other work (or yields cleanly), reducing needless ready↔waiting churn.
 4. **I/O and CPU scheduling co-design** — e.g., SPN (Shortest Process Next) favors short bursts, keeping I/O-bound jobs cycling fast and CPU busy servicing the queue.
-

Q3. Memory Bottleneck Debugging

Reworking the numbers with context:

- Code: 5 MB (shared, read-only typically — often mapped once regardless of thread count)
- Heap: 150 MB (shared across all threads of the process — this is the big one)
- Stack: 1 MB × 25 threads = 25 MB (private per-thread)
- **Total per process ≈ 180 MB**

Against 2 GB (2048 MB) RAM, one process alone isn't the problem — **so the real question the exam wants you to reason about is what "page faults" implies about *access pattern*, not raw footprint.**

Deeper root-cause reasoning:

1. **Working set exceeds resident frames:** even though total footprint is 180MB, if 25 threads are all actively touching different, scattered regions of that 150MB heap concurrently (poor locality), the OS can't keep all their "hot" pages resident — especially if multiple such processes run

simultaneously, or if RAM is shared with OS/cache/buffers, shrinking effective free memory well below 2GB.

2. **False sharing / cache-heap thrashing:** threads touching adjacent memory regions can cause pages to be evicted and reloaded repeatedly if the replacement algorithm doesn't recognize a shared working set.
3. **Excessive thread stack overhead relative to actual need:** $25 \text{ threads} \times 1\text{MB stacks} = 25\text{MB}$ is usually fine, but if this is on a memory-constrained embedded or VM-limited system, this compounds the pressure.
4. **Poor page replacement policy:** if using a naive FIFO/global-LRU without per-thread/per-process working-set awareness, actively-used pages get evicted in favor of pages that will be needed again soon — a classic thrashing symptom at smaller scale.

Two optimisations, elaborated:

1. **Thread pooling + reduce active thread count:** Instead of spawning 25 raw OS threads (each with dedicated stack + kernel scheduling overhead + potential page-table pressure), use a bounded thread pool (e.g., 8 threads = number of CPU cores) with a task queue. This reduces both memory footprint and reduces the "working set" the OS must keep resident simultaneously.
2. **Memory locality optimization + working-set-aware paging:** Restructure heap allocation using **memory pools / arena allocators** so related data used by the same thread is co-located in fewer pages (improving spatial locality and reducing the number of distinct pages touched). Combine this with a **working-set or LRU-based replacement policy with prefetching**, and consider increasing per-process resident set size (RSS) limits if the OS is being overly conservative.

Q4. Starvation Debugging

Deeper mechanism:

Multilevel Queue (MLQ) scheduling (as opposed to Multilevel *Feedback* Queue) uses **fixed, non-overlapping priority queues** — a lower-priority queue is *only* serviced when *all* higher-priority queues are completely empty. This is different from time-slicing across queues.

Given:

- Q1 (Interactive, quantum=5ms) — highest priority, likely continuously fed by user interactions

- Q2 (System, quantum=10ms) — second priority, fed by OS/system tasks
- Q3 (Batch, FCFS) — lowest priority, **no preemption within it, no time bound**

If Q1 and Q2 collectively **never go empty** (which is realistic — interactive systems almost always have *some* pending interactive/system work), Q3 gets **zero CPU time**, indefinitely. This is **indefinite postponement / starvation**, distinct from deadlock — the process P5 is not blocked or waiting on a resource cycle, it's simply never *selected* by the scheduler.

Why this is worse than it sounds: Even a single stray interactive click every few seconds is enough to keep Q1 "non-empty" often enough that Q3 essentially never gets serviced, because MLQ re-checks from the top every time it needs to pick the next process.

Optimisation, elaborated — Aging:

Aging tracks how long each process has waited and gradually increases its effective priority the longer it waits:

$$\text{effective_priority}(P5) = \text{base_priority}(Q3) + (\text{wait_time} / \text{aging_factor})$$

Once P5's waited time crosses a threshold (say 500ms), the scheduler **temporarily promotes it** to Q2 or even Q1 for one scheduling cycle, guaranteeing it eventually gets serviced — regardless of how busy the higher queues stay.

Alternative / complementary optimisation — Guaranteed CPU-share MLFQ:

Instead of strict priority, allocate a **guaranteed percentage of CPU time to each queue** (e.g., 50% Q1, 30% Q2, 20% Q3) using something like a **lottery scheduler** or **stride scheduling**. Even under heavy Q1/Q2 load, Q3 (and thus P5) is guaranteed its 20% share — converting hard starvation into, at worst, slower-but-bounded progress.

Q5. Context-Switch Overhead Debugging

Quantifying the overhead:

With a 1ms quantum and 2ms average burst:

- Most threads need **≥ 2 quantum expirations** just to finish one burst → each burst potentially causes 1–2 preemptive switches even without I/O.

- **Very high I/O wait frequency** means threads are *also* frequently switching out mid-burst to block on I/O — compounding the quantum-driven switches.
- 20,000 switches/sec × (typical switch cost of ~1–10 microseconds, plus cache/TLB flush effects that can cost much more indirectly) can consume a significant fraction of total CPU cycles purely on OS bookkeeping rather than user work.

Why small quantum + high I/O-wait is a "worst case" combination:

A quantum smaller than burst time causes **excessive preemptive switching** because the CPU rarely lets a thread finish its natural burst. Simultaneously, high I/O-wait frequency means threads *voluntarily* yield the CPU often too. The system is essentially paying full context-switch cost (saving/restoring registers, flushing pipelines, TLB misses, cache cold-start) far more often than the useful work (2ms bursts) justifies.

Two optimisations, elaborated:

1. **Increase the quantum** to be closer to or slightly above the average burst time (e.g., 3–5ms). This lets most threads complete their burst within a single quantum, cutting quantum-driven switches roughly in half or more, while still keeping responsiveness reasonable for short bursts.
2. **I/O-aware / multilevel feedback scheduling with switch coalescing:** Prioritize threads that just returned from I/O (since they tend to need very short CPU bursts before blocking again — classic I/O-bound behavior), reducing the "penalty" of frequent switching by making each switch more likely to be immediately productive. Additionally, consider **asynchronous I/O (e.g., io_uring, epoll-based event loops)** so threads don't need a full context switch every time they issue an I/O request — the kernel can service I/O completion without a full save/restore cycle for each blocked thread.

Q6. Semaphore Synchronization Debugging — Full Trace

Walking through the actual hang scenario, step by step:

Assume buffer starts empty: empty=0 is unusual as an initial value (normally empty=BufferSize=10, full=0 at start) — but here we treat the given values as a **snapshot during execution** where the buffer has become full (empty=0, full=10).

Scenario — buffer is full, both producer and consumer arrive:

1. Producer runs: `wait(mutex)` → succeeds, `mutex` now 0. Producer now tries `wait(empty)` → `empty=0`, so it **blocks**, but critically **it still holds mutex**.
2. Consumer runs: `wait(mutex)` → **blocks immediately** because `mutex=0` (held by the blocked producer).
3. Consumer can never reach `wait(full)` → consume → `signal(empty)`, which is the *only* thing that could wake the producer.
4. **Deadlock**: Producer waits for empty (which only Consumer can signal), but Consumer can't even start because Producer is holding mutex.

This is a textbook illustration of why semaphore *acquisition order* matters: mutual exclusion semaphores (`mutex`) must always be acquired *after* resource-counting semaphores (`empty/full`), never before — otherwise you create exactly this kind of circular hold-and-wait.

Correct version, with reasoning for each line:

Producer:	Consumer:
<code>wait(empty) // check space</code>	<code>wait(full) // check item available</code>
<code>wait(mutex) // then lock</code>	<code>wait(mutex) // then lock</code>
produce item	consume item
<code>signal(mutex) // unlock first</code>	<code>signal(mutex) // unlock first</code>
<code>signal(full) // then signal</code>	<code>signal(empty) // then signal</code>

By checking the **counting semaphore first**, a process only takes `mutex` when it already knows it can make progress (`space/item` is available) — so it will never hold `mutex` while blocked indefinitely on the other semaphore.

Deadlock condition violated: Specifically **hold-and-wait** (holding `mutex` while waiting on `empty`) combined with **circular wait** (Producer waits on Consumer's signal, Consumer waits on Producer's `mutex` release). Fixing the acquisition order removes the hold-and-wait condition entirely for this pair.

Q7. Thrashing — Elaborated

The thrashing feedback loop, explained:

More processes admitted → less memory per process →
more page faults → more time spent on disk I/O per process →
less CPU time actually used → OS sees "low CPU utilisation" →
OS (mistakenly) admits MORE processes to "keep CPU busy" →
even less memory per process → even more faults → ...

This is the classic **Denning's working-set thrashing curve**: CPU utilisation rises with multiprogramming degree up to a point, then **collapses** once the degree exceeds what physical memory can support — because faulting overhead dominates.

Reading the given numbers:

- 250 faults/sec is high for 50 processes (5 faults/sec/process average, but likely concentrated on a subset)
- 18% CPU utilisation — the vast majority of time is *not* spent computing
- 95% disk utilisation — nearly all disk bandwidth is consumed by paging traffic (not useful file I/O)
- 2% free frames — the system has almost no slack to accommodate any process's working set growth

The page-fault–CPU relationship, more rigorously:

Each page fault requires: trap to OS → find a free frame (or evict one, possibly writing it back to disk) → schedule a disk read → block the faulting process → context switch to something else → (eventually) resume. Disk I/O is **orders of magnitude slower** than CPU/memory access (milliseconds vs nanoseconds), so a high fault rate means processes spend most of their "wall clock" time blocked, not computing — hence CPU utilisation crashes even though the system feels "busy" (disk-bound).

Two optimisations, elaborated:

1. **Reduce multiprogramming degree via medium-term scheduling (swapping out processes):**
Temporarily suspend/swap out some of the 50 processes to disk, freeing frames for the remaining ones so their working sets actually fit in RAM. Use the **working-set model** — Δ (window size) — to determine how many frames each process truly needs, and admit only as many processes as frames allow.
2. **Local/per-process page replacement + increase physical memory:** Switch from a global replacement policy (where one process's faults can steal frames from another, spreading the

thrashing) to **local replacement**, so each process's frame allocation is protected from others. Combine with adding more physical RAM if feasible, or implementing a **Page-Fault Frequency (PFF)** algorithm that dynamically grows/shrinks a process's frame allocation based on its observed fault rate.

Q8. File System Performance — Elaborated

Why 120 seeks per request for a 500MB file is alarming:

If a 500MB file were stored **contiguously**, reading it should require close to **1 seek** (to locate the start) plus mostly sequential reads. 120 seeks per request implies the file's blocks are **heavily fragmented** across the disk — every time the read head jumps between non-contiguous blocks, it pays a full seek + rotational latency penalty (often several milliseconds each on spinning disks). $120 \times \text{even } 5\text{ms/seek} = 600\text{ms}$ of pure seek overhead per request, before any actual data transfer.

Compounding factors:

- **92% disk utilisation** means the disk is nearly saturated — new requests queue up behind existing ones, adding wait time on top of seek time.
- **15% cache hit ratio** is very low — meaning 85% of accesses go all the way to physical disk instead of being served from fast memory cache. For a database workload this often indicates either a cache that's too small relative to the working set, or an access pattern (e.g., large sequential scans) that doesn't benefit from typical LRU caching.

Two optimisations, elaborated:

1. **Defragmentation / contiguous (extent-based) allocation**: Reorganize file storage so large files are written in large contiguous extents rather than scattered blocks. Modern filesystems (ext4, XFS) use **extent-based allocation** specifically to minimize this — allocating large chunks at once rather than block-by-block, which drastically cuts seeks-per-request for large files.
2. **Cache tuning + read-ahead + SSD migration**: Increase buffer cache size and enable **sequential read-ahead** (prefetching the next blocks before they're requested, since database file scans are often at least partially sequential) to raise the hit ratio above 15%. For the highest-impact fix, migrating hot data to **SSD storage** eliminates mechanical seek latency almost entirely, turning the 120-seeks problem into a near non-issue (SSD seek cost is $\sim 1000\times$ lower than HDD).

Q9. Thread Scheduling — Elaborated

Reading the symptom pattern:

- 5,000 threads, but only 60% CPU utilisation → the CPU is **not** the bottleneck (if it were, we'd see near-100% CPU utilisation with high response time).
- 92% memory usage → memory is under heavy pressure, likely close to triggering swapping.
- 12-second response time despite CPU headroom → time is being lost to **overhead that isn't "useful work"**: thread creation/teardown, scheduling churn, memory allocation/contention, and possibly the beginnings of swap activity.

Why thread-per-request scales badly:

Each thread requires:

- A dedicated **stack** (often 1MB+ default on many OSes) → at 5,000 threads that's 5GB+ just in stacks, explaining the 92% memory usage.
- **Kernel scheduling data structures** — more threads mean more overhead for the scheduler to manage context switches, run queues, and priority calculations.
- **Creation/teardown cost** — spawning and destroying an OS thread involves kernel calls, memory allocation, and TLS (thread-local storage) setup — non-trivial overhead repeated on *every single request*.

At high request rates, this creation/teardown cost and memory pressure (potentially triggering swap-related delays) dominate response time, even though the CPU itself isn't fully saturated — the system is bottlenecked on management overhead and memory contention, not raw compute.

Two optimisations, elaborated:

1. **Thread pool (bounded worker pool)**: Pre-create a fixed number of threads (e.g., matched to CPU core count × small multiplier) and reuse them across requests via a task queue. This eliminates the per-request creation/teardown cost entirely and caps memory usage predictably, regardless of request volume.
2. **Event-driven / async I/O architecture (e.g., epoll, io_uring, or an async runtime)**: Replace the thread-per-request model with a small number of threads running an event loop that multiplexes

thousands of concurrent connections using non-blocking I/O. This is how high-performance servers (nginx, Node.js) achieve high concurrency with minimal thread count — memory usage per connection drops dramatically (no dedicated stack needed), and there's no thread creation overhead per request at all.

Q10. Disk Scheduling — Elaborated with Full Working

Given: Requests = 98, 183, 37, 122, 14, 124, 65, 67 | Head starts at 53

FCFS (as processed in arrival order):

Move	From	To	Distance
1	53	98	45
2	98	183	85
3	183	37	146
4	37	122	85
5	122	14	108
6	14	124	110
7	124	65	59
8	65	67	2

Total head movement = $45+85+146+85+108+110+59+2 = 640$ cylinders

Notice the erratic pattern: the head goes all the way up to 183, then crashes down to 37, then back up to 122, down to 14, up to 124... this back-and-forth "seeking" is exactly why average seek time is high and throughput is low — the head is doing far more total travel than the requests actually require.

SSTF (Shortest Seek Time First) — always pick nearest pending request:

Step	Current	Nearest remaining	Distance
1	53	65	12

Step	Current	Nearest remaining	Distance
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
7	183	37	146
8	37	14	23

Total = 12+2+31+24+2+59+146+23 = 299 cylinders — less than half of FCFS.

Note the flaw: SSTF greedily chases nearby requests (53→65→67→98→122→124→183) and leaves **14 and 37 stranded** until the very end, forcing one huge 146-cylinder jump back. This illustrates SSTF's known weakness — it can **starve far-away requests** indefinitely if new nearby requests keep arriving.

SCAN (elevator algorithm) — sweep in one direction, service everything, then reverse:

Moving upward from 53 toward the high end (say up to 199, then reverse):

53 → 65 → 67 → 98 → 122 → 124 → 183 → (reverse) → 37 → 14

Move	Distance
53→65	12
65→67	2
67→98	31
98→122	24
122→124	2
124→183	59
183→37 (reverse direction)	146

Move	Distance
37→14	23

Total = 299 cylinders (same as SSTF in this particular case, since the request layout happens to align with a natural sweep) — **but SCAN guarantees no request waits longer than one full sweep of the disk**, unlike SSTF which has no such bound.

Why SCAN/C-LOOK is the "suitable" recommendation over both:

- **vs FCFS:** 299 vs 640 — more than 2x improvement, because the head moves in a disciplined, monotonic sweep instead of erratically jumping around.
- **vs SSTF:** Equal total distance here, but SCAN provides a **fairness guarantee** — every request gets serviced within one sweep cycle, avoiding the starvation risk SSTF has for edge-of-disk requests (like 14, which SSTF services last, similar to SCAN, but SSTF could in principle delay it indefinitely if closer requests kept arriving — SCAN cannot).
- **C-LOOK** (a refinement of SCAN) only travels as far as the *last actual request* in each direction (rather than sweeping all the way to the disk's physical end), saving additional unnecessary travel — making it the most commonly used variant in real systems (e.g., Linux's deadline/CFQ-inspired schedulers borrow from this idea).

Final recommendation: Replace FCFS with **C-LOOK**, since it combines SCAN's fairness guarantee with SSTF-like efficiency, avoiding both the erratic back-and-forth of FCFS and the starvation risk of pure SSTF — directly addressing the "very high average seek time" and "low throughput" complaints in the log.

Rubrics for Calculation

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation